

Aufgabe 1: Ankreuzfragen (24 Punkte)

1) Einfachauswahlfragen (16 Punkte)

Bei den Einfachauswahlfragen in dieser Aufgabe ist jeweils nur **eine** richtige Antwort eindeutig anzukreuzen. Auf die richtige Antwort gibt es die angegebene Punktzahl.

Wollen Sie eine Antwort korrigieren, streichen Sie bitte die falsche Antwort mit drei waagrechten Strichen durch (~~☒~~) und kreuzen die richtige an.

Lesen Sie die Frage genau, bevor Sie antworten.

a) Gegeben seien die folgenden Präprozessor-Makros:

```
#define SUB(a, b) a - b
```

```
#define MUL(a, b) a * b
```

Was ist das Ergebnis des folgenden Ausdrucks? `4 * MUL(SUB(3,5), 2)`

2 Punkte

- ☐ -2
☐ -16
☐ 16
☐ 2

b) Welche Aussage zu Zeigern (*pointer*) ist richtig?

2 Punkte

- ☐ Eine call-by-value Übergabesemantik kann in C nur durch Zeiger abgebildet werden.
☐ Zeiger können in C nicht als Parameter an Funktionen übergeben werden.
☐ Ein Zeiger kann zur Manipulation von schreibgeschützten Datenbereichen verwendet werden.
☐ Bei Verwendung eines ungültigen Zeigers erkennt die MMU die ungültige Adresse bei der Adressumsetzung und löst einen Trap aus.

c) Welche Aussage zum Thema Unix Systemaufrufe (*system calls*) ist richtig?

2 Punkte

- ☐ Der Systemaufruf `fork()` lädt eine Programmdatei in einen neu erzeugten Prozess.
☐ Der Systemaufruf `_exit()` beendet einen laufenden Prozess und kehrt nie zurück.
☐ Der Systemaufruf `execv()` kehrt niemals zurück.
☐ Systemaufrufe werden im Userspace ausgeführt – es ist kein Wechsel in den Systemkern (*kernel*) nötig.

d) Was versteht man unter virtuellem Speicher?

2 Punkte

- ☐ Adressierbarer Speicher in dem sich keine Daten speichern lassen, weil er physikalisch nicht vorhanden ist.
☐ Speicher, der einem Prozess durch entsprechende Hardware (MMU) und durch Ein- und Auslagern von Speicherbereichen vorgespiegelt wird, aber möglicherweise größer als der verfügbare physikalische Hauptspeicher ist.
☐ Speicher, der nur im Betriebssystem sichtbar ist, jedoch nicht für einen Anwendungsprozess.
☐ Unter einem virtuellen Speicher versteht man einen physikalischen Adressraum, dessen Adressen durch eine MMU vor dem Zugriff auf virtuelle Adressen umgesetzt werden.

e) Wie wird erkannt, dass eine Seite eines virtuellen Adressraums gerade ausgelagert ist?

2 Punkte

- ☐ Im Seitendeskriptor wird ein spezielles Bit geführt, das der MMU zeigt, ob eine Seite eingelagert ist oder nicht. Falls die Seite nicht eingelagert ist, löst die MMU einen Trap aus.
☐ Das Betriebssystem erkennt die ungültige Adresse vor Ausführung eines Maschinenbefehls und lagert die Seite zuerst ein bevor ein Fehler passiert.
☐ Die MMU erkennt bei der Adressumsetzung, dass die physikalische Adresse ungültig ist und löst einen Trap aus.
☐ Bei Programmen, die in virtuellen Adressräumen ausgeführt werden sollen, erzeugt der Compiler speziellen Code, der vor Betreten einer Seite die Anwesenheit überprüft und ggf. die Einlagerung veranlasst.

f) Welche Aussage zum Thema nachrichten-basierten (*message-based*) *Inter-Process Communication (IPC)* ist richtig?

2 Punkte

- ☐ Eine Nachricht kann immer nur an einen einzigen Empfängerprozess gerichtet sein.
☐ Bei einem eingehenden Signal geht ein blockierter (*blocked*) Empfängerprozess sofort in den Zustand laufend (*running*) über, um das Signal so zügig wie möglich zu bearbeiten.
☐ *Pipes* werden immer bidirektional genutzt, sodass alle Prozesse sowohl schreibend als auch lesend darauf zugreifen.
☐ Eine Signalhandler Routine (*signal handler routine*) kann durch ein anderes eintreffendes Signal unterbrochen werden.

g) Welche Aussage zum Thema Synchronisation ist richtig?

2 Punkte

- ☐ Durch den Einsatz von Semaphore kann ein wechselseitiger Ausschluss erzielt werden.
- ☐ Einseitige Synchronisation (*unilateral synchronisation*) erfordert immer Betriebssystem-Unterstützung.
- ☐ Die V-Operation kann auf einem Semaphor nur von dem Thread aufgerufen werden, der zuvor auch die P-Operation aufgerufen hat.
- ☐ Ein Semaphor kann ausschließlich für mehrseitige Synchronisation (*multilateral synchronisation*) verwendet werden.

h) Welche Aussage zu Unterbrechungen (*interrupts*) ist richtig?

2 Punkte

- ☐ Bei der mehrfachen Ausführung eines unveränderten Programms mit gleicher Eingabe treten Unterbrechungen immer an den gleichen Stellen auf.
- ☐ Eine Signalleitung zeigt dem Prozessor eine Unterbrechung an. Der Prozessor sichert den aktuellen Zustand bestimmter Register und springt eine vordefinierte Behandlungsfunktion (*interrupt handler*) an.
- ☐ Die Behandlung von Unterbrechungen kann nur mittels Spezialhardware deaktiviert werden.
- ☐ Eine Signalleitung zeigt dem Prozessor eine Unterbrechung an. Der Prozessor bricht die gerade bearbeitete Maschineninstruktion ab und führt einen neuen Prozess aus.

2) Mehrfachauswahlfragen (8 Punkte)

Bei den Mehrfachauswahlfragen in dieser Aufgabe sind jeweils m Aussagen angegeben, davon sind n ($0 \leq n \leq m$) Aussagen richtig. Kreuzen Sie alle richtigen Aussagen an. Jede korrekte Antwort in einer Teilaufgabe gibt einen Punkt, jede falsche Antwort einen Minuspunkt. Eine Teilaufgabe wird minimal mit 0 Punkten gewertet, d. h. falsche Antworten wirken sich nicht auf andere Teilaufgaben aus.

Wollen Sie eine falsch angekreuzte Antwort korrigieren, streichen Sie bitte das Kreuz mit drei waagrechten Strichen durch (~~☒~~).

Lesen Sie die Frage genau, bevor Sie antworten.

a) Gegeben sei folgendes Programm:

4 Punkte

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 const double PI = 3.1415;
5
6 int test(void) {
7     static int a;
8     int b;
9     a++;
10
11     printf("%d\n", b);
12
13     PI = a * PI;
14     b = a + PI;
15
16     return a;
17 }
```

Welche der folgenden Aussagen bzgl. dieses Programms sind korrekt?

- ☐ Der Inhalt der Datei *stdio.h* wird vor dem Übersetzen an die Stelle der entsprechenden *include*-Anweisung einkopiert.
- ☐ Der Aufruf von *printf* in Zeile 11 gibt immer den Wert 0 auf *stdout* aus, da die Variable *b* automatisch mit *NULL* initialisiert ist.
- ☐ Beim Überschreiben der Variable *PI* in Zeile 13 tritt ein Compilerfehler auf, weil *const* Variablen nicht überschrieben werden dürfen.
- ☐ Die globale Variable *PI* ist initialisiert und liegt daher im BSS Segment des Adressraums.
- ☐ Die globale Variable *PI* ist nicht in anderen Modulen sichtbar.
- ☐ Die Variable *a* ist uninitialisiert und enthält daher einen zufälligen Wert.
- ☐ Die Variable *a* behält ihren Wert über mehrere Aufrufe der Funktion *test* hinweg.
- ☐ Die Variable *b* liegt im Stacksegment des Adressraums.

b) Welche der folgenden Aussagen zum *Translation Lookaside Buffer* (TLB) sind richtig?

4 Punkte

- ☐ Der TLB ist ein Cache der immer die FIFO-Strategie zur Ersetzung von Einträgen implementiert.
- ☐ Wird eine Speicherabbildung im TLB nicht gefunden, wird der auf den Speicher zugreifende Prozess mit einer Schutzraumverletzung (*segmentation fault*) abgebrochen.
- ☐ Verändert sich die Speicherabbildung von virtuellen auf physikalische Adressen aufgrund eines Kontextwechsels (*context switch*), so werden auch die Daten im TLB ungültig.
- ☐ Der TLB ist ein sehr schneller Cache der MMU, der physikalische in virtuelle Adressen umsetzt.
- ☐ Der TLB ist ein voll-assoziativer Cache, der zur Adressumsetzung genutzt wird.
- ☐ Der TLB ist ein sehr schneller Cache der MMU, der virtuelle in physikalische Adressen umsetzt.
- ☐ Wird eine Speicherabbildung im TLB nicht gefunden, wird die Abbildung in den Seitentabellen (*page tables*) nachgeschlagen und im TLB eingetragen.
- ☐ Verändert sich die Speicherabbildung von virtuellen auf physikalische Adressen aufgrund eines Kontextwechsels (*context switch*), bleiben alle Einträge im TLB trotzdem gültig.

Aufgabe 2: pass (Parallel Synchronized Serving Machine) (45 Punkte)

Sie dürfen diese Seite zur besseren Übersicht bei der Programmierung heraustrennen!

Schreiben Sie ein POSIX-1.2008 und C11-konformes Programm `pass`, welches einen Datenbankserver implementiert, der parallel Anfragen bearbeitet. Anfragen werden von einem eigenen Thread vom Standardeingabekanal (*stdin*) eingelesen und in einer FIFO-Queue (verkettete Liste) verwaltet. Eine durch die Konstante `WORKERS` vorgegebene Anzahl an Arbeiterthreads entnimmt die Anfragen aus der Liste und kümmert sich um die Ausführung der Datenbankoperationen. Die Kommunikation zwischen den Threads soll mittels modulglobaler Variablen geschehen.

Das Programm soll folgendermaßen strukturiert sein:

– Funktion **int** `main(void)`:

Es werden zunächst alle relevanten Strukturen initialisiert. Danach werden der Annahmethread und die Arbeiterthreads gestartet. Anschließend wartet der Hauptthread **passiv** auf die Beendigung des Annahmethreads, sodass dessen Ressourcen freigegeben werden können. Nachdem sich der Annahmethread beendet hat, signalisiert der Hauptthread den Arbeiterthreads, dass sich diese beenden sollen, sobald keine offenen Anfragen mehr ausstehen. Der Hauptthread wartet ebenfalls passiv auf die Beendigung der Arbeiterthreads. Sobald sich alle Threads beendet haben, wird das gesamte Programm beendet.

– Funktion **void** `*receiver_thread(void *arg)`:

Der Annahmethread liest Datenbankankfragen mittels der Funktion `readLine()` von *stdin*. Der `readLine()`-Aufruf blockiert solange bis eine gültige Anfrage vorliegt und gibt diese zurück. Neue Anfragen werden in einer modulglobalen Queue den Arbeiterthreads zur Verfügung gestellt. Nutzen Sie zur Implementierung der Queue die `job_t`-Struktur. Wird eine neue Anfrage in die Queue aufgenommen, wird dies den Arbeiterthreads geeignet signalisiert.

Sobald der Annahmethread durch das Ende der Eingabe oder einen Fehler **NULL** von der Funktion `readLine()` zurückgegeben bekommt, beendet er sich. Eine Fehlerbehandlung von `readLine()` ist nicht nötig.

– Funktion **void** `*worker_thread(void *arg)`:

Die Arbeiterthreads warten passiv auf das Eintreffen neuer Datenbankankfragen. Wird ein neuer Auftrag verfügbar, wird das `job_t * Element` aus der Queue entnommen und die Anfrage mithilfe der Funktion `databaseExecute()` ausgeführt. Arbeiterthreads beenden sich, sobald der Annahmethread sich beendet hat und keine Anfragen mehr in der Liste sind.

Ein zentraler Bestandteil dieser Aufgabe ist die **korrekte Synchronisation** mithilfe der vorgegebenen, **passiv wartenden Semaphor-Implementierung** – aktives Warten ist nicht erlaubt. Langsame Funktionen (z.B. `printf(3)`) dürfen nicht in kritischen Abschnitten ausgeführt werden.

Hinweise:

- Nutzen Sie eine Kombination aus leerer Queue und Signalisierung mittels Semaphore um die Arbeiterthreads zuverlässig zu beenden.
- Ihnen steht das aus der Übung bekannte Semaphor-Modul zur Verfügung.
- Die Implementierung der Queue erfolgt in den Funktionen `receiver_thread()` und `worker_thread()` und nicht in separaten Funktionen.
- Die Funktion `readLine()` steht Ihnen durch die Schnittstelle `tools.h` zur Verfügung. Die Funktion `databaseExecute()` wird durch die Schnittstelle `database.h` bereit gestellt.
- Beschreibungen der Schnittstellen `sem.h`, `tools.h` und `database.h` und der jeweiligen Funktionen finden Sie in den angehängten Manpages.
- Es ist **keine** Fehlerbehandlung/`fflush()` für Ausgaben auf `stdout` und `stderr` nötig. Achten Sie ansonsten auf korrekte und vollständige Fehlerbehandlung.

```
#include <errno.h>
#include <pthread.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "database.h"
#include "tools.h"
#include "sem.h"

#define WORKERS 7

static void die(char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

// Funktions- & Strukturdekl., globale Variablen, etc.

struct job {

};

typedef struct job job_t;
```

```
// Funktion main

// Initialisierungen

// Annahmethread starten

// Arbeiterthreads starten
```

```
// Auf Beendigung warten
```

5

```
// Ausführungsende an Arbeiterthreads signalisieren
```

3

```
// Aufräumen
```

7

```
// Ende Funktion main
```

M:

```
// Funktion receiver_thread
```

```
// Anfragen einlesen und Job erstellen
```

```
// Job in Queue einhängen
```

10

```
// Ende Funktion receiver_thread
```

R:

```
// Funktion worker_thread
```

R:

```
// Job aus Queue entnehmen
```

```
// Anfrage ausführen
```

W:

Vervollständigen Sie das folgende Makefile. Es soll ein Target `pass` unterstützt werden, welches das Programm `pass` baut. Greifen Sie dabei stets auf Zwischenprodukte (z.B. `pass.o`) zurück. Gehen Sie davon aus, dass die vorgegebenen Module `sem.o`, `tools.o` und `database.o` stets vorliegen und daher nicht erzeugt werden müssen.

Nutzen Sie dabei die Variablen CC und CFLAGS konventionskonform. Achten Sie darauf, dass das Makefile ohne eingebaute Variablen und Regeln (Aufruf von `make -Rr`) funktioniert!

.PHONY: all clean

CC=gcc

```
CFLAGS=-std=c11 -pedantic -Wall -D_XOPEN_SOURCE=700
```

```
all: pass
```

clean:

```
rm -f pass pass.o
```

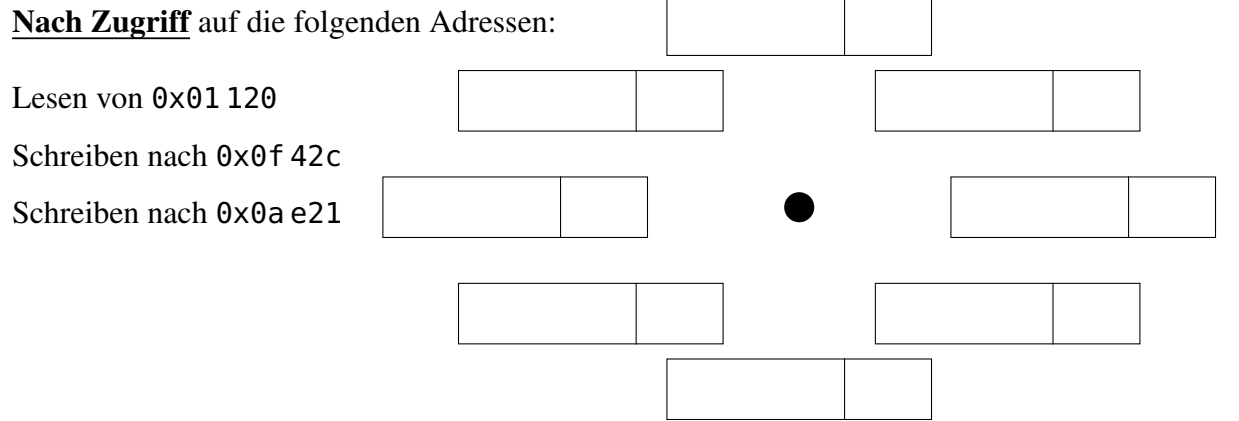
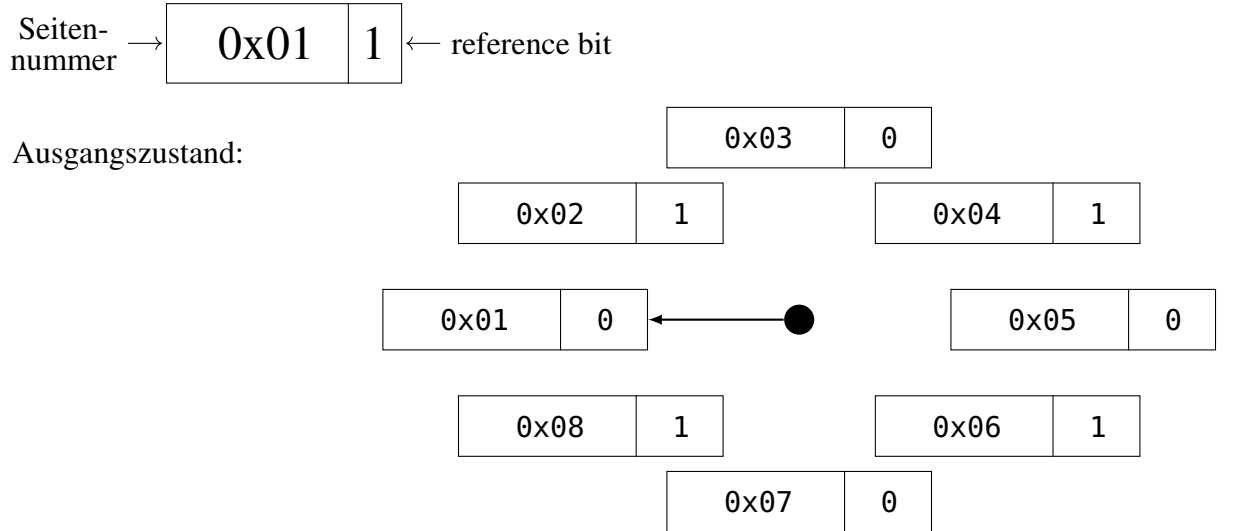
Mk:

Aufgabe 3: Speicherverwaltung (8 Punkte)

1) Eine in der Praxis gut einsetzbare Seitenersetzungsstrategie ist Second Chance: CLOCK. In dieser Aufgabe soll CLOCK als (prozess-)lokale Seitenersetzungsstrategie eingesetzt werden. (5 Punkte)

Im Folgenden sind die für die Seitenverwaltung erforderlichen Daten der aktuell anwesenden Seiten eines Prozesses dargestellt. Nehmen Sie eine Seitengröße von 4096 Bytes an (ergibt 12 Bit Offset). Gehen Sie davon aus, dass alle Seiten les- und schreibbar sind und alle zugegriffenen Adressen gültig sind.

Hinweis: Der CLOCK-Zeiger zeigt jeweils auf den Eintrag, der bei der nächsten Suche nach einem freien Seitenrahmen (*page frame*) als erstes überprüft wird.

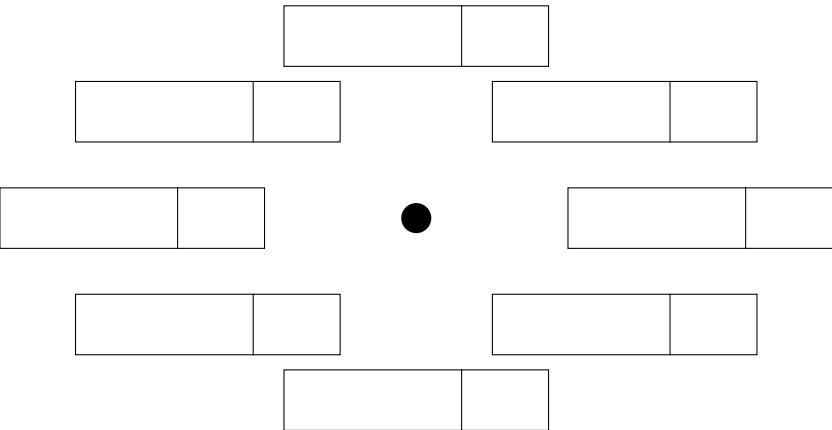


2) Nehmen Sie an, ein System verwendet die CLOCK Strategie. Welches Phänomen tritt auf, wenn alle Referenzbits der Seitendeskriptoren auf 1 gesetzt sind und anschließend nur noch auf nicht-eingelagerte Seiten zugegriffen wird? (1 Punkt)

3) Falls kein freier Seitenrahmen im Hauptspeicher verfügbar ist, muss eine Seite ausgelagert werden. Nennen Sie zwei weitere mögliche Strategien zur Bestimmung der zu verdrängenden Seite (ausgenommen Second Chance, CLOCK). Beschreiben Sie die Strategien kurz. (2 Punkte)

Ersatzgrafik für Teilaufgabe 3.1.

Sie dürfen diese Grafik nutzen, falls Sie sich beim Ausfüllen der Grafik in 3.1. verzeichnet haben. Markieren Sie eindeutig, welche der Grafiken zur Bewertung herangezogen werden soll!



Aufgabe 4: Scheduling (13 Punkte)

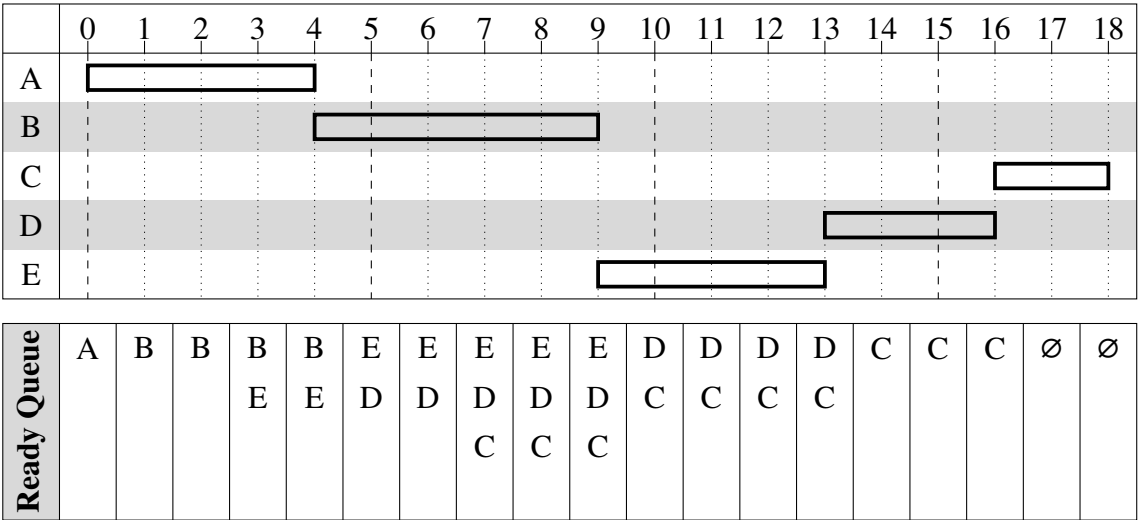
1) In einem Ein-Prozessor System (*single-processor system*) sollen fünf rechenintensive Prozesse mit den folgenden Zeiten ausgeführt werden:

Prozess	arrival time	service time
A	0	4
B	1	5
C	7	2
D	5	3
E	3	4

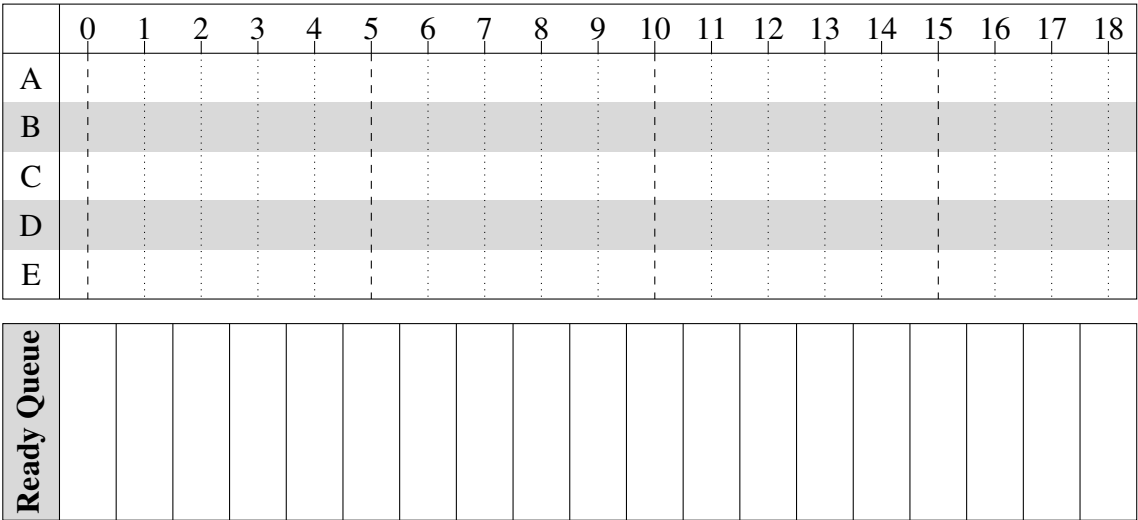
Arrival time bezeichnet die Ankunftszeit, also den Zeitpunkt, an dem der Prozess rechenbereit (*ready*) wird. Die **service time** ist die Bedien- bzw. Rechenzeit, die der Prozess benötigt um seine Ausführung abzuschließen.

Zeichnen Sie in dem untenstehenden Diagramm für die gegebene Scheduling-Strategie ein, welcher Prozess zum jeweiligen Zeitpunkt aktiv (*running*) ist. Die Ready Queue kann als Gedankenstütze zur leichteren Bearbeitung der Aufgabe genutzt werden, wird jedoch nicht bepunktet.

Beispiel First-Come First-Served (FIFO)



Round Robin (RR) mit Zeitquantum $q = 2$ (9 Punkte)



2) Worin unterscheiden sich kooperatives (*cooperative*) und präemptives (*preemptive*) Scheduling? Nennen Sie zusätzlich je einen Vorteil der beiden Schedulingarten. (4 Punkte)

Ersatzgrafik für Teilaufgabe 4.1.

Sie dürfen diese Grafik nutzen, falls Sie sich beim Ausfüllen der Grafik in 4.1. verzeichnet haben. Markieren Sie eindeutig, welche der Grafiken zur Bewertung herangezogen werden soll!

